

This is a copy of a shared ChatGPT conversation
Report conversation

Release Readiness.md mentions a Feedback mechanism for beta testers. Will we still need that if I use TestFlight?

I would still keep it, but I'd narrow its scope considerably.

TestFlight solves **distribution** and gives you **crash reports, screenshots, and basic feedback**, but it doesn't replace having an intentional way to collect useful product feedback.

I'd think of it this way:

Need	TestFlight	In-app feedback	
Install beta	✓	✗	
Automatic crash reports	✓	✗	
Screenshot with annotation	✓	✗	
General comments	✓	✓	
Feature requests	⚠ Possible, but unstructured	✓ Better	
Repro steps for bugs	⚠ Depends on tester	✓ You can guide them	
Analytics on common issues	✗	✓ if you choose	

For your MVP, where the testers are mostly friends and people you've invited personally, I actually think the simplest solution is best.

I'd update the document to something like:

Beta Feedback

TestFlight will be the primary feedback mechanism during the initial beta. Testers can submit feedback directly through TestFlight, including annotated screenshots and crash information.

The app does not require an in-app feedback feature for the initial MVP. If beta testing reveals that structured bug reports or feature requests would be valuable, an in-app feedback screen (or integration with GitHub Issues, a simple web form, or similar) can be considered after the first round of testing.

I'd actually go one step further

Since your goals are:

- polish the app
- learn AI-assisted development

- build a portfolio project

I would *not* spend engineering time building feedback infrastructure.

Instead, create a simple Google Form or similar with questions like:

- What were you trying to do?
- What happened?
- What did you expect?
- How severe is the problem?
- Device model
- iOS version

Then include the link in your TestFlight invitation email (or send it separately to testers). That's about 30 minutes of work and gives you much more consistent reports than an in-app form.

Once the app is public on the App Store, you can decide whether an in-app "Send Feedback" feature is worth adding. Many successful indie apps don't have one—they rely on App Store reviews, support email, or a website contact form.

So if I were editing `Release Readiness.md`, I'd change "Implement feedback mechanism for beta testers" to something like:

Use TestFlight for beta feedback. Provide testers with a simple external feedback form. Do not build an in-app feedback feature unless beta testing demonstrates a clear need.

That feels much more in line with the MVP philosophy we've been following for NextSeason.

 Uploaded a file

Here's the latest version of the file - please update it as you suggest

I like this direction because it makes the document tell two stories at once:

1. **What you've already accomplished** (which is valuable for portfolio reviewers).
2. **What still needs to happen before release** (which is what you need day-to-day).

I've created a reorganized version with:

- A new **Completed Improvements** section near the top that summarizes the major work already finished.
- A clearer separation between completed work and remaining release tasks.
- The beta feedback item updated to recommend **TestFlight + an external feedback form**, with an in-app feedback feature explicitly deferred.

You can download it here:

 **Release Readiness_reorganized.md**

One thing I'd like to do in a future pass is make the document read more like a professional engineering artifact rather than a running checklist. We could organize it into sections like:

-  Completed
-  Required Before Portfolio Release
-  Recommended Before App Store Release
-  Future Enhancements (with a pointer to `Roadmap.md`)

I think that would be a very polished document to include in the GitHub repository and would reinforce the engineering discipline you've shown throughout this project. </>

Ok, we'll do that. I think the last thing I need to work on before beta is Analytics - do you want to add any details to that section before I hand it off to Cursor/Claude?

Yes. In fact, I think analytics is the one place where we can get a lot of value from a relatively small amount of code.

The biggest trap with analytics is recording everything because you can. For an MVP, I'd only record events that answer questions you'll actually ask after your beta.

Here's what I'd recommend.

Core Analytics (Required)

Search

- Search performed
 - Query length (not the query itself, for privacy)
 - Number of results
 - Search duration

Questions answered:

- Are people searching successfully?
 - Is TVMaze returning poor results?
 - Is search fast enough?
-

Search Result Interaction

- Search result selected
- Added to watchlist directly from search
- Removed from watchlist directly from search

Questions:

- Do people use the quick-add button?
 - Are they opening details first?
-

Show Detail

Record:

- Detail screen viewed
- Added to watchlist
- Removed from watchlist

Questions:

- How many people inspect details before adding?
 - Is the detail page actually useful?
-

Watchlist

Record:

- Watchlist viewed
- Watchlist item opened
- Watchlist item removed

Questions:

- Do users actually come back to their watchlist?
 - How often do they clean it up?
-

Notifications

Record:

- Notification permission granted/denied
- Notification tapped
- App opened from notification

Questions:

- What percentage allow notifications?
 - Are notifications actually bringing users back?
-

Nice to Have

Empty States

Record when users see:

- No search results
- Empty watchlist

These often point to UX issues.

Errors

Record non-fatal errors such as:

- TVMaze timeout
- Decoding failure
- Notification scheduling failure

Don't just log them to the console.

I Would NOT Record

- Actual search strings (privacy)
- Show names searched for
- Actor taps
- Every button press
- Scroll position
- Screen time

That kind of instrumentation generates a lot of noise and almost never changes product decisions for an app of this size.

One Event I'd Add

We discussed this briefly before, and I still think it's worth doing.

If someone taps an actor's name today and nothing happens...

...record it.

That tells you whether users *expect* actor navigation.

No UI.

No alert.

No feedback.

Just:

```
actor_name_tapped
```



If five beta testers do it once, ignore it.

If 70% of users do it repeatedly, you've just discovered one of your highest-value post-MVP features.

That's exactly the sort of thing analytics should uncover.

Implementation

I would also keep the implementation extremely simple:

```
AnalyticsService.track(.watchlistViewed)
```



```
AnalyticsService.track(.searchPerformed(resultCount: 12))
```

```
AnalyticsService.track(.notificationOpened)
```

where the events are a Swift enum with associated values.

That way you can start with logging to the console during development, swap in a backend later (such as Firebase or another analytics provider), and leave the rest of the app unchanged.

One addition I'd make to the Release Readiness document

I'd add a success criterion to the Analytics section:

Goal: Collect enough anonymous usage data during beta testing to answer product questions and prioritize post-MVP development, rather than measuring engagement or marketing metrics.

I really like that framing because it explains *why* analytics exists in this app. You're not trying to optimize ad revenue or retention; you're trying to learn what real users actually do so you can build the right features next. That fits perfectly with both the MVP and the portfolio story.

That all sounds good - please update the document

Done! I expanded the Analytics section to reflect what we discussed, while keeping it focused on MVP and beta learning rather than "track everything."

The updated section now includes:

- **A clear goal statement** explaining *why* analytics exists.
- **A concrete list of events to instrument**, including:
 - Search activity (without recording search text)
 - Watchlist interactions
 - Show detail usage
 - Notification permissions and opens
 - Empty states
 - Non-fatal errors
 - The "actor name tapped" event to discover unmet user expectations
- **Design notes** covering:
 - Privacy (no search strings or PII)
 - Avoiding excessive instrumentation
 - Using an `AnalyticsService` abstraction so you can swap analytics providers later without touching the rest of the app.

You can download it here:

 **Release Readiness_analytics.md**

I think this is a strong handoff for Cursor/Claude. It's specific enough that it can implement the instrumentation consistently, but it leaves the implementation details (event enum, provider, dependency injection, etc.) up to the AI and your architectural preferences. </>